Generated benchmark artifacts

Typeset from `paper/generated/x86_64/` by `benchmarks/scripts/build_pdf.py`. Every number resolves to a row in the result set named below; see `PROVENANCE.md` for the inputs, transformation and caveats of each artifact.

```
result set: current
commit: 00d8c08aa656
host: a2
measured: 2026-08-09
inputs: sha256:c858a66763006796
```

Dataset	Tool	Metric	-@	Mapped	Mapq 60	Missed (%)	Wrong Q60	Index (s)	Map (s)	Mem (GB)
B01	shmap-rs	Containment	1	149 194	139 309	6.78	—	7.91	33.17	2.52
B01	shmap-rs	Jaccard	1	146 120	138 421	7.37	—	8.01	47.41	2.55
B01	shmap-rs	bucket_SH	1	149 236	138 250	7.49	—	6.90	25.03	2.66
B01	cpp-shmap	Containment	1	149 194	139 309	6.78	—	—	—	18.85
B01	cpp-shmap	Jaccard	1	146 120	138 421	7.37	—	—	—	18.85
B01	cpp-shmap	bucket_SH	1	149 236	138 250	7.49	—	—	—	18.85
B01	mapquik	—	32	48 913	—	—	—	—	19.50	19.37
B01	winnowmap2	—	32	189 022	—	—	—	—	1328.30	9.78
B02	shmap-rs	Containment	1	125 000	117 532	5.97	0	7.65	25.45	2.49
B02	shmap-rs	Jaccard	1	125 000	119 065	4.75	6	7.13	38.87	2.79
B02	shmap-rs	bucket_SH	1	125 000	116 800	6.56	1	6.94	22.33	2.82
B02	cpp-shmap	Containment	1	125 000	117 532	5.97	—	—	—	18.85
B02	cpp-shmap	Jaccard	1	125 000	119 065	4.75	—	—	—	18.85
B02	cpp-shmap	bucket_SH	1	125 000	116 802	6.56	—	—	—	18.85
B02	mapquik	—	32	39 965	—	—	—	—	19.20	19.38
B02	winnowmap2	—	32	129 488	—	—	—	—	1006.40	9.40
B03	shmap-rs	Containment	1	241 991	227 521	6.19	—	6.84	40.29	2.57
B03	shmap-rs	Jaccard	1	241 265	228 887	5.63	—	8.27	55.46	2.45
B03	shmap-rs	bucket_SH	1	242 050	226 931	6.43	—	8.57	34.59	2.49
B03	cpp-shmap	Containment	1	241 991	227 521	6.19	—	—	—	18.85
B03	cpp-shmap	Jaccard	1	241 265	228 886	5.63	—	—	—	18.85
B03	cpp-shmap	bucket_SH	1	242 050	226 934	6.43	—	—	—	18.85
B03	mapquik	—	32	89 559	—	—	—	—	20.90	19.42
B03	winnowmap2	—	32	280 093	—	—	—	—	997.00	8.04
B04	shmap-rs	Containment	1	2 419 796	2 273 122	6.28	—	7.36	397.88	2.59
B04	shmap-rs	Jaccard	1	2 411 461	2 286 085	5.74	—	7.29	543.45	2.60
B04	shmap-rs	bucket_SH	1	2 420 331	2 267 229	6.52	—	7.24	373.36	2.51
B04	cpp-shmap	Containment	1	2 419 796	2 273 121	6.28	—	—	—	18.85
B04	cpp-shmap	Jaccard	1	2 411 461	2 286 083	5.74	—	—	—	18.85
B04	cpp-shmap	bucket_SH	1	2 420 331	2 267 238	6.52	—	—	—	18.85
B04	mapquik	—	32	897 326	—	—	—	—	44.80	19.36
B04	winnowmap2	—	32	2 808 979	—	—	—	—	9019.50	8.38
B05	shmap-rs	Containment	1	39 608	37 839	58.97	—	7.18	14.34	2.53
B05	shmap-rs	Jaccard	1	5 956	5 409	94.13	—	7.15	17.50	2.63
B05	shmap-rs	bucket_SH	1	41 115	39 226	57.46	—	7.19	13.65	2.47
B05	cpp-shmap	Containment	1	39 608	37 839	58.97	—	—	—	18.85
B05	cpp-shmap	Jaccard	1	5 956	5 409	94.13	—	—	—	18.85
B05	cpp-shmap	bucket_SH	1	41 115	39 225	57.47	—	—	—	18.85
B05	winnowmap2	—	32	99 107 [2]	—	—	—	—	331.50	6.15

Table 1: Long-read mapping tools on the benchmark suite. Mapq 60 is the count of confident mappings; Missed is the share of input reads either unmapped or below

Dataset	Metric	Matches per read			Potential		Buckets per read	
		possible	examined	in mapping	realized	unrealized	seeded	final
B01	Containment	113 462	2 376	206	47.8×	11.6×	194.0	2.38
B01	Jaccard	113 462	2 376	203	47.8×	11.7×	194.0	2.31
B01	bucket_SH	113 462	2 376	208	47.8×	11.4×	194.0	2.28
B02	Containment	113 804	2 907	209	39.2×	13.9×	206.2	2.29
B02	Jaccard	113 804	2 907	208	39.2×	13.9×	206.2	2.29
B02	bucket_SH	113 804	2 907	211	39.2×	13.8×	206.2	2.26
B03	Containment	70 179	1 886	117	37.2×	16.1×	251.8	2.35
B03	Jaccard	70 179	1 886	117	37.2×	16.1×	251.8	2.34
B03	bucket_SH	70 179	1 886	119	37.2×	15.9×	251.8	2.24
B04	Containment	69 463	1 938	116	35.8×	16.6×	250.3	2.35
B04	Jaccard	69 463	1 938	116	35.8×	16.7×	250.3	2.34
B04	bucket_SH	69 463	1 938	118	35.8×	16.4×	250.3	2.25
B05	Containment	67 358	307	51	219.6×	6.0×	84.8	0.99
B05	Jaccard	67 358	307	9	219.6×	33.1×	84.8	0.17
B05	bucket_SH	67 358	307	54	219.6×	5.7×	84.8	1.00

Table 2: Efficiency of the seed heuristic. Realized potential is possible matches divided by matches examined (work avoided); unrealized potential is matches examined divided by matches inside the reported mapping (work still wasted).

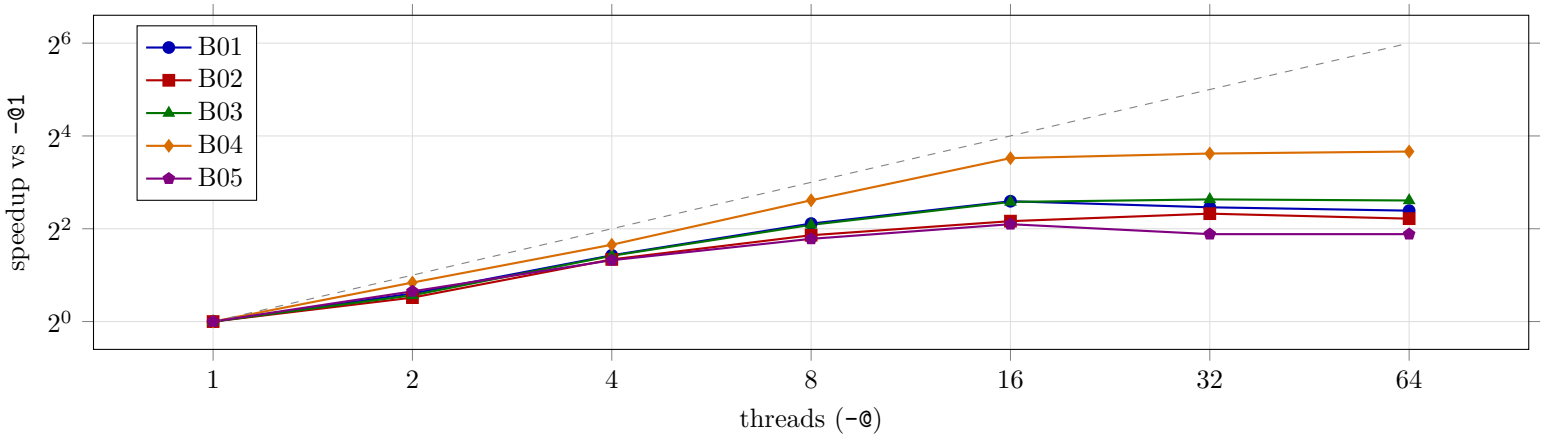

Figure 1: Whole-run speedup against thread count, Containment. The dashed line is linear speedup.

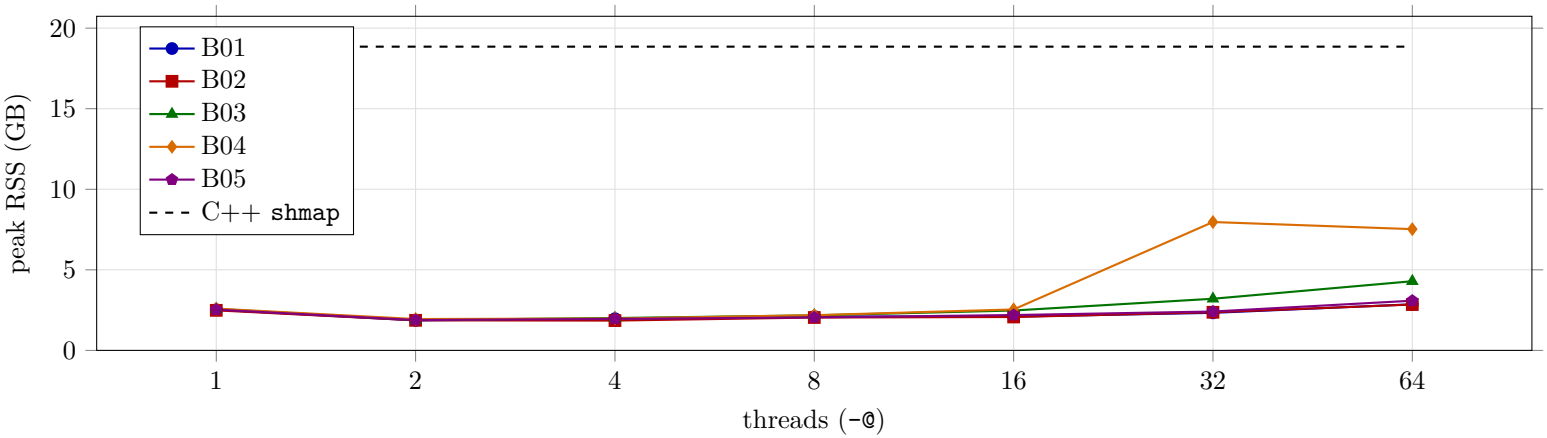

Figure 2: Peak resident memory against thread count, Containment, with the C++ reference as a horizontal line.

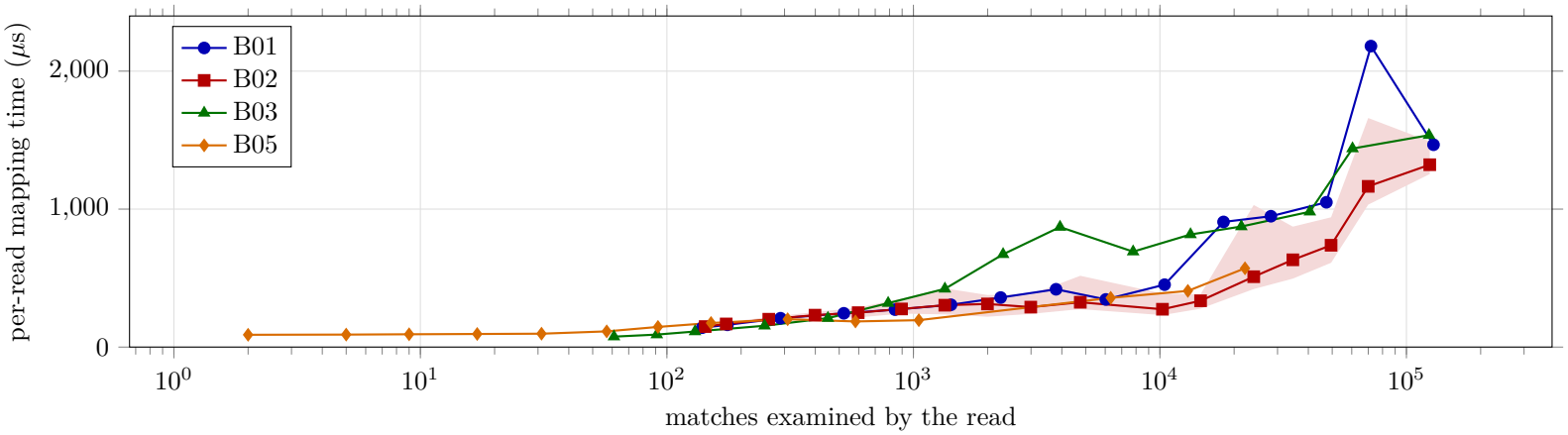

Figure 3: Per-read mapping time against the number of matches the read examined, Containment. Points are per-bin medians over reads grouped into 18 logarithmic bins; the shaded band is the interquartile range for the simulated set.

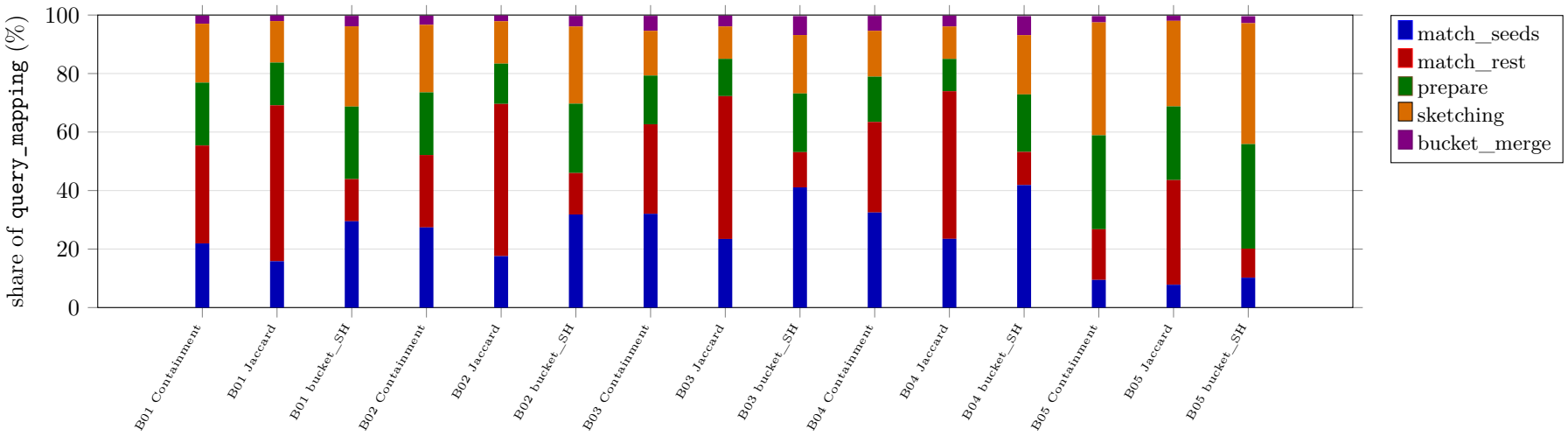

Figure 4: Where mapping time goes: stage shares of `query_mapping`, single-threaded.